Gradient Boosting

Gradient boosting builds a prediction model **sequentially**. Each new learner is trained to improve the current model, and its contribution is added to the running prediction. For squared-error regression, “improve” has a particularly simple form: fit the new learner to the current residuals. For other loss functions, fit it to the **negative gradient** of the loss with respect to the current predictions.

This gives one framework for regression and classification. The base learners are often small decision trees, but the central idea is the stagewise additive update.

1. An additive model, built one step at a time

Let $F_m(x)$ be the model after m boosting steps. A new weak learner $h_m(x)$ is added to the current model:

$$F_m(x) = F_{m-1}(x) + \nu h_m(x),$$

where $\nu > 0$ is the **learning rate** (also called shrinkage). After M steps,

$$F_M(x) = F_0(x) + \nu \sum_{m=1}^{M} h_m(x).$$

The learners are not fitted independently. Each h_m depends on the current predictions F_{m-1}, so changing an earlier step changes what later learners are asked to fit.

With regression trees as base learners, the final model is a sum of small trees. Each tree can make a modest adjustment in different parts of the predictor space. The sum can therefore be much more flexible than any one small tree.

Contrast with other ensembles

Method	How the base learners are fit	How predictions are combined
Bagging / random forest	Trees are fit separately, often to bootstrap samples	Average predictions or vote

Method	How the base learners are fit	How predictions are combined
AdaBoost	Learners are fit sequentially using reweighted observations	Weighted vote or score
Gradient boosting	Learners are fit sequentially to reduce the current loss	Add shrunken learner predictions

The key feature of gradient boosting is the **additive correction to the model's predictions**. It does not generally reweight observations in the AdaBoost manner.

2. Regression: fit the residuals

Suppose the loss is half the squared error for observation i:

$$\ell(y_i, F(x_i)) = \frac{1}{2}\big(y_i - F(x_i)\big)^2.$$

At the current model F_{m-1}, the derivative of this loss with respect to its prediction is

$$\frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)} = F(x_i) - y_i.$$

The negative derivative is therefore

$$-\frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)} = y_i - F(x_i),$$

which is the residual. So fitting a learner to residuals is a gradient-descent step in prediction space: the residual points in the direction that locally decreases squared error most quickly.

For squared-error regression, a common starting prediction is the sample mean:

$$F_0(x) = \bar{y}.$$

Then repeat:

1. Calculate residuals $r_{im} = y_i - F_{m-1}(x_i)$.
2. Fit a small tree h_m to the pairs (x_i, r_{im}).
3. Update the predictions by $F_m(x) = F_{m-1}(x) + \nu h_m(x)$.

For this loss, the negative gradient and the residual are exactly the same. With another loss, the corresponding negative gradient is often called a **pseudo-residual**.

A small regression example

Suppose four observations have $y = (1, 2, 5, 6)$, and a tree is allowed one split at $x = 2.5$. Start at the mean, $F_0(x) = 3.5$, for every observation. The first residuals are $(-2.5, -1.5, 1.5, 2.5)$. Fitting a stump means predicting the average residual within each of its two leaves.

i	x_i	y_i	$F_0(x_i)$	$r_{i1} = y_i - F_0(x_i)$	$h_1(x_i)$	$F_1(x_i), \nu = 0.5$
1	1	1	3.5	-2.5	-2	2.5
2	2	2	3.5	-1.5	-2	2.5
3	3	5	3.5	1.5	2	4.5
4	4	6	3.5	2.5	2	4.5

The stump predicts the mean residual in each leaf: $(-2, -2, 2, 2)$. With learning rate 0.5, the update is

$$F_1(x_i) = 3.5 + 0.5 h_1(x_i).$$

The new residuals are $(-1.5, -0.5, 0.5, 1.5)$. A second stump using the same split predicts $(-1, -1, 1, 1)$, the mean residual in each leaf. After the second update,

$$F_2(x_i) = F_1(x_i) + 0.5 h_2(x_i) = (2, 2, 5, 5),$$

leaving residuals $(-1, 0, 0, 1)$. The model has reduced the residuals, but it has not driven every residual to zero. Shrinkage makes each step smaller, and the trees' leaf structure limits the adjustments they can make.

3. The general gradient-boosting step

The residual trick extends to a differentiable loss $\ell(y, F)$. At boosting step m, compute one negative gradient for each training case using the current prediction:

$$r_{im} = - \left. \frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)} \right|_{F = F_{m-1}}.$$

These r_{im} values are the pseudo-responses for that step. Fit a base learner $h_m(x)$ to predict them from x, then add a shrunken version of that learner:

$$F_m(x) = F_{m-1}(x) + \nu h_m(x).$$

The tree does not usually reproduce every pseudo-response exactly. It approximates the negative-gradient values using a small number of regions. That approximation is what makes the procedure a **functional** or **prediction-space** gradient descent: the update is a function of x, not a step in a fixed finite vector of coefficients.

In regression with squared error, $r_{im} = y_i - F_{m-1}(x_i)$. For other objectives—such as absolute error, a robust loss, or classification log loss—the pseudo-responses change to match that objective. Some implementations also optimize a separate step value within each leaf; the simple update above describes the core idea.

4. Binary classification: gradients of log loss

For a binary response $y_i \in \{0, 1\}$, let the model output a score $F(x)$ on the log-odds scale. Convert the score to a probability with the logistic function:

$$p(x) = \frac{1}{1 + e^{-F(x)}}.$$

The binary log loss for one observation is

$$\ell(y, F) = -y \log p - (1 - y) \log(1 - p).$$

Differentiating with respect to the score F gives

$$\frac{\partial \ell(y, F)}{\partial F} = p - y, \qquad r = -\frac{\partial \ell(y, F)}{\partial F} = y - p.$$

Thus the pseudo-residual is the difference between the observed label and the current predicted probability. It is **not** generally the raw label minus a class prediction of 0 or 1. A learner is fit to these pseudo-residuals, and its shrunken output updates the score F. The updated score is then transformed back to a probability with the logistic function.

For example, if $y = 1$ and the current probability is $p = 0.7$, the pseudo-residual is $1 - 0.7 = 0.3$: the score should move upward. If $y = 0$ and $p = 0.7$, it is $0 - 0.7 = -0.7$: the score should move downward. A large magnitude indicates a larger local correction under log loss.

For a constant initial score under binary log loss, the loss-minimizing starting value is the log odds of the training prevalence:

$$F_0 = \log\left(\frac{\bar{y}}{1-\bar{y}}\right), \qquad \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i.$$

At prediction time, the score $F_M(x)$ can be turned into a probability $p(x)$; classify using a chosen threshold if a class label is needed. The default threshold of 0.5 is a decision rule, not part of the gradient-boosting update, and another threshold may be appropriate when error costs or class prevalence make the costs unequal.

5. What controls the fit?

The main choices for tree-based gradient boosting include:

Choice	Role	Typical effect of increasing it
Number of boosting steps M	How many corrections are added	More opportunities to reduce training loss; validation performance can eventually stop improving
Learning rate ν	Size of each correction	Larger, faster updates; can need fewer trees and may fit too aggressively
Tree complexity	Size of patterns each learner can capture, often controlled by depth or leaf count	More complex corrections; can fit interactions and local structure but may raise overfitting risk
Subsampling, if used	Fraction of training cases used to fit each tree	Adds randomness and can regularize; the best fraction is data dependent

The learning rate and number of steps interact. A smaller ν usually calls for more trees to reach a comparable training fit, and can improve generalization when paired with enough stages. It also costs more computation. More trees do not automatically cause worse test performance at every step, so choose the stopping point using validation or cross-validation rather than a rule based only on the training loss.

To compare settings, keep the validation procedure and performance metric appropriate to the problem. For classification, for example, accuracy alone may not reflect the goal when classes are imbalanced or the costs of false positives and false negatives differ. Keep the final test set out of choices about tree depth, learning rate, number of trees, or classification threshold. Fit any data-dependent preprocessing within the training folds to avoid leakage.

6. Algorithm summary

Given training cases (x_i, y_i), differentiable loss ℓ, number of steps M, and learning rate ν:

1. Choose an initial constant prediction F_0 that minimizes the total loss over constants.
2. For $m = 1, \ldots, M$:
   - Compute pseudo-residuals

$$r_{im} = - \left. \frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{m-1}}.$$

   - Fit a small learner h_m to predict r_{im} from x_i.
   - Update $F_m(x) = F_{m-1}(x) + \nu h_m(x)$.
3. Return $F_M(x)$, optionally transforming it to a probability or other target scale dictated by the loss.

For squared-error regression, this reduces to repeatedly fitting residuals. For binary log-loss classification, it fits the current probability errors $y_i - p_i$ and updates the log-odds score.

Summary

- Boosting builds a model sequentially by adding small learners to the current prediction.
- Under squared-error regression, the negative gradient is the ordinary residual $y_i - F(x_i)$.
- Under a different loss, the negative gradient supplies pseudo-residuals; classification therefore does not usually mean fitting raw residuals from hard class labels.
- The learning rate shrinks each update. Its interaction with tree complexity and number of steps controls the model's flexibility.
- Select complexity and stopping point using validation, then reserve a separate test set for final assessment.

Sources: James, Witten, Hastie, Tibshirani, and Taylor, *An Introduction to Statistical Learning: With Applications in Python*, second edition (2023), §§8.2.3 (pp. 347–350) and 8.3.4; Jerome H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” *The Annals of Statistics* 29(5), 1189–1232 (2001), doi:10.1214/aos/1013203451. The numerical regression example is constructed for these notes.